

Operational Agents

Carry out bounded work with approved tools and evidence of the result.

Bring the customer's task. This playbook supplies reusable execution controls.

How to use this conversation

Each module moves through the same three steps:

Step	Customer question
Discuss	What can the agent do, and when must it stop?
Decide	Where should policy, state, approval, and recovery live?
Prove	Which observable result shows the boundary held?

Record the choice, owner, open question, and evidence gate as you go. Keep runtime detail in the scenario modules.

Define the work before adding an agent

A useful explanation does not prove that an operation completed.

Choose the task boundary and the conditions that require a stop.

Choose how much to delegate

Option	Decision
Read-only	The agent can inspect approved state.
Approved mutation	A person must approve the exact proposed effect.
Fixed workflow	Use when the next step needs no model judgment.

Prove the boundary

Run an allowed read and an out-of-scope request.

Inspect the destination. A denied request must produce no effect.

Connect the customer's execution environment

Identify the application runtime and its approved model and tool endpoints. Prove the runtime identity can reach them. Local checks inspect the reference engine; they do not connect the customer's operation.

Choose the environment

Use the customer's approved environment first. Add only missing resources through its normal deployment process. Keep the synthetic reference backend separate from the tenant integration.

Prove runtime access

Call the selected model and perform an authorized read of the approved test record. Test a denied identity too. A synthetic backend response is not tenant acceptance.

A schema is only the first boundary

The application validates arguments.

The destination service must also enforce access and decide whether the requested update is still valid.

Choose a narrow operation set

Separate reads from proposed writes.

Require an expected record version and an operation ID for updates. Avoid arbitrary commands and unrestricted update objects.

Prove destination guarantees

Reject unexpected arguments.

Replay the same operation ID and confirm one effect. Reuse that ID with different arguments; the destination must reject it.

Conversation history is not approval

A new process must know what is pending without asking the model to remember.

Persist progress outside the conversation.

Put information where it belongs

Information	Owner
Policy	Approved knowledge source
Current facts	Live system
Pending work and budget	Application task store
Optional durable preferences	Explicit memory design

Restart before approving

Open the task from a fresh process.

The proposal digest and prior evidence must match. The destination must still be unchanged.

Resume must preserve limits

An interrupted caller must not grant another unrestricted budget.

Count work across the task's lifetime.

Choose stop and retry rules

Bound model turns and tool attempts.

Retry only suitable reads. A write timeout requires reconciliation because the operation may already have committed.

Exhaust the budget deliberately

Confirm the task stops and retains prior results.

Exhausted does not mean rolled back. Inspect effects before starting replacement work.

Approval belongs to one proposal

Show the exact tool and arguments to the reviewer.

A changed proposal or record version needs a new decision.

Choose the approval boundary

The local CLI demonstrates the contract using a trusted operator.

A customer pilot needs authenticated approvers and policy rules outside the model.

Test approval failure paths

Deny a proposal and inspect the unchanged record.

Then change an approved proposal's arguments or expected version. The old decision must no longer authorize execution.

A lost reply is an unknown result

The destination can commit before the caller records success.

Do not equate interruption with failure.

Reconcile before deciding

Destination evidence	Response
Matching receipt	Record the result and continue.
Contradictory receipt	Stop and investigate.
No authoritative result	Keep the task unresolved.

Interrupt after commit

Restart the process and read the operation ledger.

Recover one receipt with one effect. A second write would fail this check.

Execution correctness and model quality differ

The application can enforce approval while the model chooses a poor next step.

Test both against the customer's task.

Choose the pilot evidence

Use local failure cases for runtime behavior.

Add live synthetic prompts, then approved customer integration checks. Assign owners to work the kit does not implement.

Review the actual outcome

Inspect operation receipts and the destination state.

Record gaps in authenticated approval, shared storage, or telemetry. Module completion alone does not establish production readiness.

Close the discussion

Before the next working session, record:

- the bounded task and explicit refusal conditions
- approved reads, proposed writes, and destination-side controls
- where task state, budgets, and operation receipts will live
- who can approve an exact proposal and how stale approval is rejected
- how operators reconcile an interrupted write
- the customer-channel evidence required before release
- one owner and due date for every unresolved decision

The next session should start from these decisions, not reopen them from memory.